

FIR Filter Project Report

Noise Removal Using a Moving-Average FIR Filter

1. Introduction

The goal of this project is to remove a high-frequency noise component from a useful signal using a simple Finite Impulse Response (FIR) filter implemented in MATLAB. The input signal is a 50 Hz sine wave (the desired signal) corrupted by a 10 kHz sine wave (the noise) and a 2.5 V DC offset. The signal is sampled at $F_s = 50$ kHz and filtered using a moving-average FIR filter. We then visualize the result in both the time and frequency domains.

2. Theoretical Background

2.1 The Input Signal

The input signal is the sum of a desired sinusoid, a noise sinusoid, and a DC bias:

$$\begin{aligned}x[n] &= 2.5 + 0.005 \cdot (x_{\text{signal}}[n] + x_{\text{noise}}[n]) \\x_{\text{signal}}[n] &= 280 \cdot \sqrt{2} \cdot \sin(2\pi \cdot 50 \cdot n \cdot T_s) \\x_{\text{noise}}[n] &= 50 \cdot \sin(2\pi \cdot 10000 \cdot n \cdot T_s)\end{aligned}$$

where $T_s = 1 / F_s = 1 / 50000$ s.

2.2 FIR Filter Equation

An FIR filter of order M computes each output sample as a weighted sum of the current and the M previous input samples:

$$y[n] = \sum_{k=0}^M b_k \cdot x[n-k]$$

In a moving-average filter, every coefficient is the same: $b_k = 1 / (M + 1)$. This means the output is just the average of the latest $M + 1$ input samples:

$$y[n] = (1 / (M + 1)) \cdot (x[n] + x[n-1] + \dots + x[n-M])$$

Averaging cancels out fast oscillations (high frequencies) while preserving slow variations (low frequencies). The filter therefore acts as a low-pass filter, which removes the 10 kHz noise but keeps the 50 Hz signal.

2.3 Filter Structure

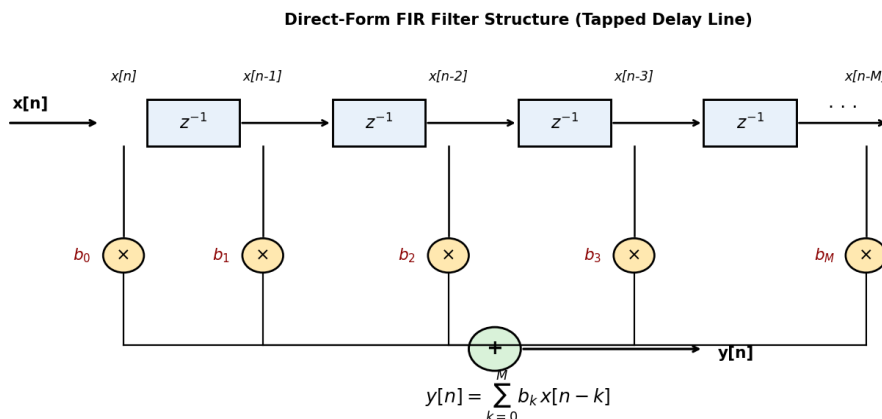


Figure 1: Direct-form structure of an FIR filter. Each z^{-1} block stores a past sample.

3. MATLAB Code Explanation by Related Code Sections

This section explains the MATLAB script in grouped blocks. Each built-in command is explained where it appears in the code, instead of listing all functions separately.

3.1 Clearing the MATLAB Environment

```
clear, close all, clc; % Clear environment
```

This line prepares MATLAB before running the project. `clear` removes old variables from the workspace so previous results do not affect the new run. `close all` closes any open figure windows. `clc` clears the Command Window to make the output easier to read.

3.2 Defining the Discrete-Time Signal Parameters

```
n = 1:2000; % Samples (n)

f_signal = 50; % Desirable Signal Frequency
f_noise = 10e3; % Noise Frequency
Fs = 50e3; % Sampling Frequency
Ts = 1/Fs; % Sampling Time
```

The vector `n = 1:2000` creates 2000 sample positions. The colon operator generates equally spaced integer values from 1 to 2000. `f_signal` is the useful low-frequency signal at 50 Hz, while `f_noise` is the unwanted high-frequency noise at 10 kHz. `Fs` is the sampling frequency, equal to 50 kHz, and `Ts` is the sampling time calculated as $1/Fs$.

The notation `10e3` means $10 \times 10^3 = 10,000$, and `50e3` means $50 \times 10^3 = 50,000$. This is scientific notation used to write large values clearly.

3.3 Generating the Desired Signal, Noise Signal, and Input Signal

```
x_signal = 280*sqrt(2)*sin(2*pi*f_signal*Ts*n); % Desirable Signal
x_noise = 50*sin(2*pi*f_noise*Ts*n); % Noise Signal

x_sn = 2.5 + 0.005*(x_signal + x_noise); % Input Signal = Desirable Signal + Noise

NN = length(x_sn); % Finding no. of samples in the input signal
```

The first equation generates the desired 50 Hz sinusoidal signal. The term $280 \times \sqrt{2}$ converts 280 V RMS into peak voltage because $V_{\text{peak}} = V_{\text{rms}} \times \sqrt{2}$. The built-in function `sqrt(2)` calculates the square root of 2.

The built-in function `sin()` creates a sinusoidal waveform. In MATLAB, `sin` uses radians, so the angle is written as $2 \times \pi \times f \times Ts \times n$. The built-in constant `pi` represents π . Multiplying by `Ts` and `n` converts the sample number into discrete time.

The second equation generates the 10 kHz noise signal. The third equation forms the final noisy input `x_sn` by adding the desired signal and the noise, scaling the result by 0.005, and shifting it upward by a 2.5 V DC offset.

The function `length(x_sn)` returns the number of samples in the vector `x_sn`. Here it stores the value 2000 in `NN` so the same number can be reused in loops and FFT calculations.

3.4 Plotting the Noisy Input Signal

```
figure
plot(n,x_sn)
grid on
xlabel('Samples (n)')
ylabel('Voltage (v)')
title('Input Signal + Noise')
```

```
h1 = findobj(gcf,'type','line');
set(h1,'linewidth',2)
```

figure opens a new figure window for the graph. plot(n,x_sn) draws the noisy input signal against the sample number. grid on adds grid lines to make the waveform easier to read.

xlabel, ylabel, and title add the x-axis label, y-axis label, and graph title. These functions do not change the data; they only improve the graph presentation.

gcf means get current figure. findobj(gcf,type,line) searches inside the current figure and finds all plotted line objects. set(h1,linewidth,2) changes the thickness of those plotted lines to make them clearer.

3.5 Initializing the FIR Filter Output and Delay Variables

```
y_vector_2 = zeros(1,NN); % Initializing the Filter Output Vector

x2_n_15 = 0;
x2_n_14 = 0;
x2_n_13 = 0;
x2_n_12 = 0;
x2_n_11 = 0;
x2_n_10 = 0;
x2_n_9 = 0;
x2_n_8 = 0;
x2_n_7 = 0;
x2_n_6 = 0;
x2_n_5 = 0;
x2_n_4 = 0;
x2_n_3 = 0;
x2_n_2 = 0;
x2_n_1 = 0;
```

zeros(1,NN) creates a row vector filled with zeros. This is called pre-allocation. It reserves memory for the FIR output before the loop starts, which makes the code faster and cleaner than increasing the vector size inside the loop.

The variables x2_n_1 to x2_n_15 represent delayed samples of the input signal. For example, x2_n_1 means x[n-1], x2_n_2 means x[n-2], and so on. At the beginning, they are set to zero because there are no previous samples before the first input sample arrives.

3.6 Calculating the FIR Filter Output Sample by Sample

```
for ii = 1:NN

    x2_n = x_sn(ii); % Get new Input sample and assign it to x(n)

    % Calculate the FIR Filter Output for M = 15
    y2_n = (1/16)*(x2_n + x2_n_1 + x2_n_2 + x2_n_3 + x2_n_4 + x2_n_5 + x2_n_6 + x2_n_7 + x2_n_8 +
    x2_n_9 + x2_n_10 + x2_n_11 + x2_n_12 + x2_n_13 + x2_n_14 + x2_n_15);

    y_vector_2(ii) = y2_n;
```

The for loop repeats the filtering operation from the first sample to the last sample. In every iteration, ii is the current sample index. x_sn(ii) reads the current noisy input sample and stores it in x2_n, which represents x[n].

The FIR output y2_n is calculated as an average of the current sample and previous samples. This is a moving-average FIR filter. Because high-frequency noise changes quickly from one sample to the next, averaging reduces it. The low-frequency 50 Hz signal changes slowly, so it remains mostly preserved.

3.7 Updating the Delay Line

```
% Update the previous samples
x2_n_15 = x2_n_14;
```

```

x2_n_14 = x2_n_13;
x2_n_13 = x2_n_12;
x2_n_12 = x2_n_11;
x2_n_11 = x2_n_10;
x2_n_10 = x2_n_9;
x2_n_9 = x2_n_8;
x2_n_8 = x2_n_7;
x2_n_7 = x2_n_6;
x2_n_6 = x2_n_5;
x2_n_5 = x2_n_4;
x2_n_4 = x2_n_3;
x2_n_3 = x2_n_2;
x2_n_2 = x2_n_1;
x2_n_1 = x2_n;

```

```
end
```

After calculating the output, the old samples are shifted by one position. This is the delay-line operation of the FIR filter. The current sample $x2_n$ becomes $x2_{n-1}$ in the next iteration, the old $x2_{n-1}$ becomes $x2_{n-2}$, and so on.

This part is very similar to a hardware FIR filter. In hardware, each $x[n-k]$ value would be stored in a register or delay block. At each clock pulse, the samples shift to the next register.

3.8 Plotting the Noisy and Filtered Signals Together

```

figure
plot(n,x_sn)
hold on
plot(n, y_vector_2,'r')
hold off
grid on
xlabel('Samples (n)')
ylabel('Voltage (V)')
title('Filtering Noise from Signal Using FIR Filter')
legend('Signal + Noise','FIR Filtered Signal')
h1 = findobj(gcf,'type','line');
set(h1,'linewidth',2)

```

This block compares the original noisy signal and the FIR filtered output on the same graph. The first plot draws the noisy signal. `hold on` tells MATLAB to keep the first graph and add the next plot on top of it. The second plot draws the filtered signal in red using `r`. `hold off` returns the figure to normal plotting behavior.

`legend` identifies the two curves, while `grid`, `xlabel`, `ylabel`, and `title` make the graph readable. The final two lines again use `gcf`, `findobj`, and `set` to increase the line width.

3.9 Computing the FFT Spectrum of the Original Signal

```

NN = length(x_sn); % Finding no. of samples in the input signal

X_k = fft(x_sn); % Finding spectrum of the input signal

Mag_X_k_double_sided = abs(X_k)/NN; % Calculating Spectrum Magnitude

Mag_X_k_single_sided = Mag_X_k_double_sided(1:NN/2);
Mag_X_k_single_sided(2:NN/2) = 2*Mag_X_k_single_sided(2:NN/2);

```

This block converts the original noisy signal from the time domain to the frequency domain. `fft(x_sn)` calculates the Fast Fourier Transform, which shows which frequencies exist inside the signal.

The FFT output X_k is complex, so `abs(X_k)` calculates the magnitude of each frequency component. Dividing by NN normalizes the spectrum according to the number of samples.

The FFT of a real signal is symmetric, so only the first half is needed for the single-sided spectrum. The amplitudes from index 2 to $NN/2$ are doubled because the removed negative-frequency half also contained the same energy. The DC component is not doubled.

3.10 Computing the FFT Spectrum of the Filtered Signal

```
Y_k = fft(y_vector_2); % Finding spectrum of the filtered signal

Mag_Y_k_double_sided = abs(Y_k)/NN; % Calculating Spectrum Magnitude

Mag_Y_k_single_sided = Mag_Y_k_double_sided(1:NN/2);
Mag_Y_k_single_sided(2:NN/2) = 2*Mag_Y_k_single_sided(2:NN/2);

freq = Fs*(0:(NN/2)-1)/NN; % Converting k into Frequency
```

This block repeats the same FFT steps for the filtered output. By comparing `Mag_X_k_single_sided` and `Mag_Y_k_single_sided`, we can see how much the FIR filter reduced the 10 kHz noise component.

The `freq` vector converts FFT bin numbers into real frequency values in Hz. The formula uses the sampling frequency `Fs` and the number of samples `NN`, so each FFT point is placed at its correct frequency.

3.11 Plotting the Single-Sided Spectrum

```
figure
plot(freq, Mag_X_k_single_sided)
hold on
plot(freq, Mag_Y_k_single_sided, '--r')
hold off
grid on
legend('Original Signal', 'Filtered Signal')
title('Single-Sided Spectrum of the signals, x(n) and x_f(n)')
xlabel('Frequency (Hz)')
ylabel('Spectrum Magnitude')
h1 = findobj(gcf, 'type', 'line');
set(h1, 'linewidth', 2.5)
```

This final plotting block displays the frequency spectrum of both signals. The original signal shows the desired low-frequency part and the unwanted 10 kHz noise. The filtered spectrum shows that the 10 kHz component is strongly reduced after filtering.

The command `--r` plots the filtered spectrum as a red dashed line. This makes it visually different from the original signal spectrum. The labels, legend, title, grid, and line-width commands improve presentation and make the result suitable for the report.

4. Results

Figure 2 shows the noisy input signal (blue) together with the filtered output (red). The 10 kHz noise that appears as the thick fuzzy band around the input is almost entirely removed, while the 50 Hz signal is preserved.

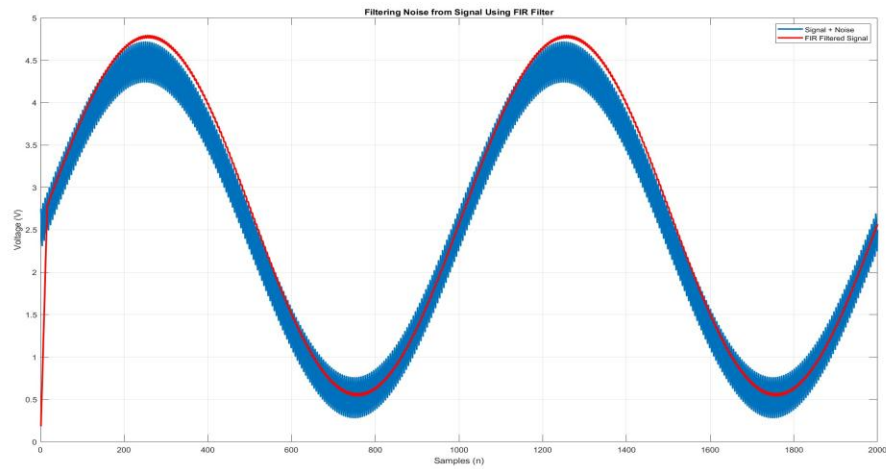


Figure 2: Time-domain comparison of the noisy and filtered signals ($M = 15$).

Figure 3 shows the single-sided amplitude spectrum. The small peak at 10 kHz in the blue (input) spectrum is the noise component; in the red (filtered) spectrum this peak is removed, while the low-frequency component (DC + 50 Hz) is preserved.

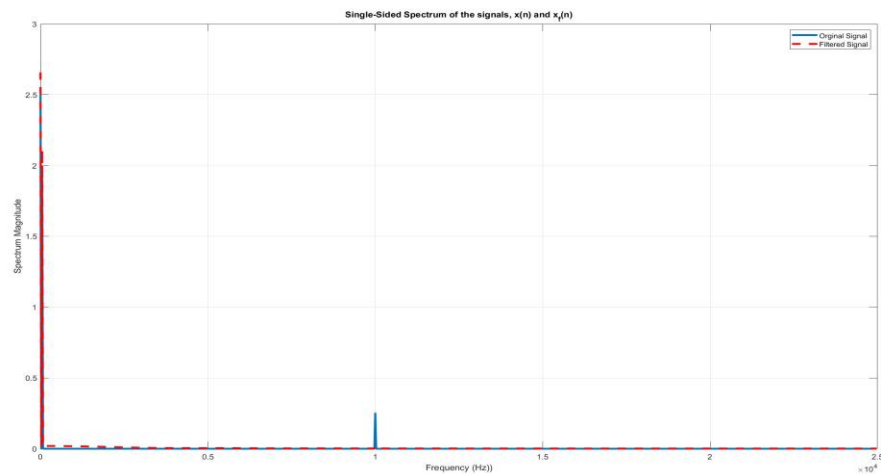


Figure 3: Frequency spectrum of the input (blue) and filtered (red) signals.

5. Conclusion

A simple moving-average FIR filter of order $M = 15$ was implemented in MATLAB as a tapped delay line. The filter successfully removed the 10 kHz noise component while preserving the desired 50 Hz signal. Both the time-domain and frequency-domain plots confirm the correct behavior of the filter. Increasing M would give stronger noise attenuation but a longer start-up transient and a slightly larger delay.

6. DSP Applications in Mechatronics

Mechatronics combines mechanical engineering, electronics, and computing to build systems such as robots, drones, CNC machines, and precision positioning stages. These systems rely on sensors whose signals are always contaminated by noise from the electronics and the environment. Digital Signal Processing (DSP), and especially digital filters, are used to clean these signals so that the control system can work correctly.

Application Example: FIR Filter for Mobile Robot Navigation

A clear example is the use of FIR filters to clean the sensor signals of a mobile robot. A mobile robot uses accelerometers, gyroscopes, magnetometers, and GPS to know its position and follow a path. These sensors produce noisy signals, and if the noise is not removed, the robot's position estimate becomes inaccurate, and it cannot follow the planned route smoothly.

a low-pass FIR filter is designed using the window method with a Hanning window. The filter specifications were:

- Sampling frequency: 300 MHz
- Cut-off frequency: 6 MHz
- Stopband attenuation: 40 dB above 75 MHz

The filter is implemented by exactly the same equation that we used in our project:

$$y[n] = \sum h_k \cdot x[n - k]$$

The only difference is the choice of coefficients: the coefficients are shaped by a Hanning window, while in our project all coefficients are equal (moving-average filter). In both cases the result is a clean signal in which the high-frequency noise has been removed while the useful low-frequency information is preserved.

A similar use of FIR filters in mechatronics is the use an FIR filter inside a repetitive controller for high-precision nanopositioning stages. This confirms that FIR filters are used in a wide range of mechatronic systems, from mobile robot navigation to nanometre-scale precision motion control.

7. Hardware Implementation

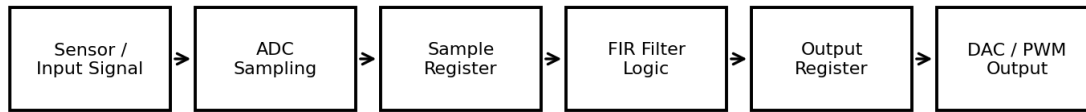
The moving-average FIR filter used in this project can be implemented directly in hardware using digital components such as registers, adders, multipliers, and a clock source. The hardware structure follows the same tapped-delay-line architecture used in the MATLAB simulation. Each new input sample enters the system through an Analog-to-Digital Converter (ADC), then passes through a sequence of delay registers that store previous samples $x[n-1]$, $x[n-2]$, ..., $x[n-15]$. At every clock pulse, all samples shift one step to the next register.

The stored samples are connected to an adder network that sums the current and previous values. The result is multiplied by the coefficient $1/16$ to produce the filtered output signal $y[n]$. Because all coefficients are equal in a moving-average filter, the hardware implementation becomes simpler than other FIR filter designs. Instead of using many different multipliers, the design can use a simple binary shift operation to divide the final sum by 16.

A practical implementation can be achieved using an FPGA such as the Xilinx Spartan-6 or Intel Cyclone series. The FPGA provides fast parallel processing, allowing all addition operations to occur simultaneously. The filtered output is then converted back into an analog signal using a Digital-to-Analog Converter (DAC) if needed for real-world applications.

This type of FIR filter hardware is commonly used in mechatronics systems, motor controllers, robotics, sensor signal conditioning, and communication systems where real-time noise removal is required.

Input → ADC → Delay Registers → Adder Network → Divide by 16 → Output



Clocked digital path: every clock pulse moves one new sample through the FIR hardware.

Figure 4: General hardware path for implementing the FIR filter.

References

- [1] A. S. Imam, L. Salisu, and D. Z. Yanmida, "Design of an FIR Digital Filter for a Mobile Robot Navigation Sensors," International Journal of Scientific & Engineering Research, vol. 12, no. 9, pp. 807–810, September 2021.

https://www.researchgate.net/publication/378775575_Design_of_an_FIR_Digital_Filter_for_a_Mobile_Robot_Navigation_Sensors/citation/download

- [2] Y. R. Teo, A. A. Eielsen, J. T. Gravdahl, and A. J. Fleming, "Discrete-Time Repetitive Control With Model-Less FIR Filter Inversion for High Performance Nanopositioning," in Proc. 2014 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), Besançon, France, July 2014, pp. 1664–1669.

<chrome-extension://efaidnbmnnnibpcaglpcllefindmkaj/https://www.precisionmechatronicslab.com/wp-content/uploads/2015/05/C14f1.pdf>